Practice NEA

Amelie Machin

May 2025

Contents

1 Analysis **1**
1.1 Problem statement 1
1.2 Dialogue 2
1.3 Existing solutions 2
1.4 Research 2
1.5 Objectives 2
1.6 Modelling 3

2 Design **6**
2.1 Databases 6
2.2 Diagrams 6

3 Technical solutions and evaluations **9**
3.1 19th May 9
3.2 4th June 16

4 Testing **24**

1 Analysis

1.1 Problem statement

At the beginning of the school year, teachers are tasked with having to learn about their students, who they may never have seen before, and design classroom layouts based on how their individual personalities may affect a learning environment. This layout may change as the teacher gets to know the students, and figures out what works best to create the most ideal classroom for students to best learn important information.

This tool aims to help teachers of primary school, secondary school, and college with both the problem of designing this ideal layout, and with being able to put names to faces sooner so that the teacher is able to recognise each student sooner in all of the different classes they teach.

Students will also be able to benefit from this tool as the classes they have will become more personalised and structured sooner, allowing them to learn more.

This tool needs to be accessible for teachers with limited sight or colour blindness, and students with special educational or physical needs must have their needs presented to teachers within this tool to allow them to be catered for in the classroom.

1.2 Dialogue

My third party...

1. What features would be helpful for you as a teacher on this tool?
2. Have you had any experience with seating plan tools in the past? What did was good and not so good about them?
3. What special needs may need to be accommodated for on a seating chart?
4. What POI would be best suited to use as a reference point to identify the orientation of the chart compared to the classroom?
5. Would you like the ability to change this POI after chart creation?
6. Would having notes or tags for each student be helpful or would it just clutter the screen? For example, a tag for people with special learning needs that don't affect seating or the ability to make a note of a warning you gave a student for poor behaviour.

1.3 Existing solutions

Laminated paper Classcharts

1.4 Research

1.5 Objectives

1. Teachers must register onto the system
   (a) They must complete a registration form where they enter their full name, email, password and confirm password
   (b) Email must be in a valid format
   (c) Password must be between 8-16 characters and contain at least one capital letter, number, and symbol
   (d) Password and confirm password must match
   (e) Password must be stored encrypted
   (f) Email must not already be registered

2. Teachers must log into the system before they can edit any class data
    (a) The email and password must match one that is stored
3. Teachers can have classes
    (a) Teachers can have multiple classes
    (b) Teachers can edit class data, such as class name, max seating capacity, and the students
    (c) Teachers can delete classes
    (d) Class data can be saved and loaded via a database
4. There must be a GUI
    (a) Seats must be visible on the GUI
        i. The layout of the seats can be changed by clicking and dragging
        ii. Each seat can only have one student assigned to it
        iii. The seats can change orientation
    (b) Students must be visible on the GUI
        i. Students are either assigned to a seat or are in a sidebar
        ii. Each student has an image, full name, and special tags where appropriate
            A. Special tags are colour coded
            B. Each student's image can be hidden or unhidden individually
            C. Each student's name can be hidden or unhidden individually
            D. Every student's name can be hidden or unhidden with the same button
            E. Every image for the class can be hidden or unhidden with the same button
        iii. Students can be dragged across the GUI
            A. Students snap to seats when left click is let go of and the student is visibly mostly over the seat
5. Information must be stored in relational databases
    (a) Databases must be in 3rd normal form
        i. The same email can only be registered once. Any attempts to re-register an email are rejected.

1.6 Modelling

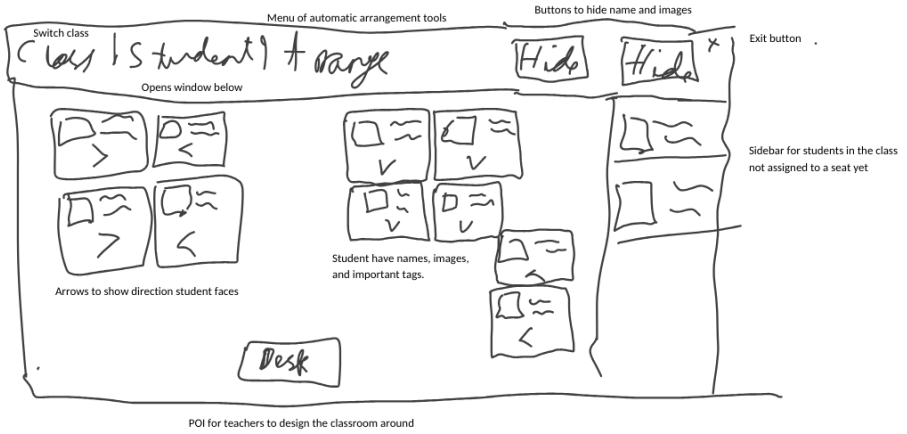


Figure 1: A sketch of the main seating chart GUI with some students arranged in the classroom

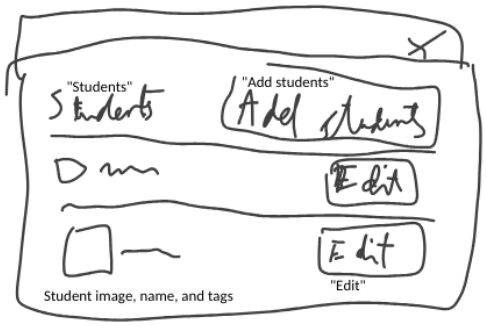


Figure 2: A sketch of the pop-up window that appears when the "Student" button is clicked

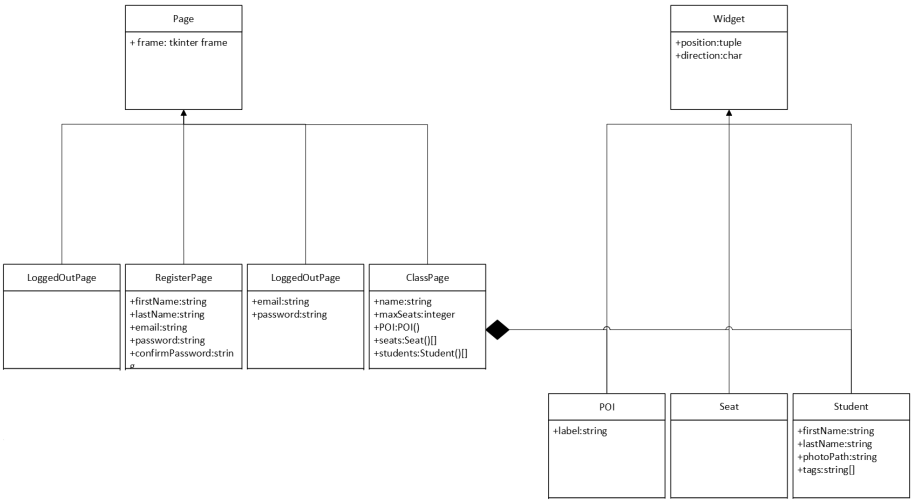


Figure 3: Class diagram showing relationships between classes and their attributes

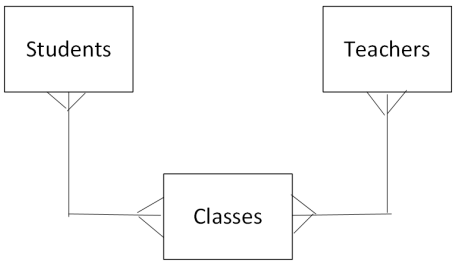


Figure 4: Entity relationship diagram between databases that have not yet been normalised

2 Design

2.1 Databases

Teachers(TeacherID, FirstName, LastName, Email, Password)
Students(StudentID, FirstName, LastName, PhotoPath)
Classes(ClassID, Name, MaxSeats)
Seats(SeatID, ClassID, xPosition, yPosition)
Tags(TagID, Name, Colour)
StudentClass(StudentID, ClassID, SeatID)
TeacherClass(TeacherID, ClassID)
StudentTags(StudentID,TagID)

2.2 Diagrams

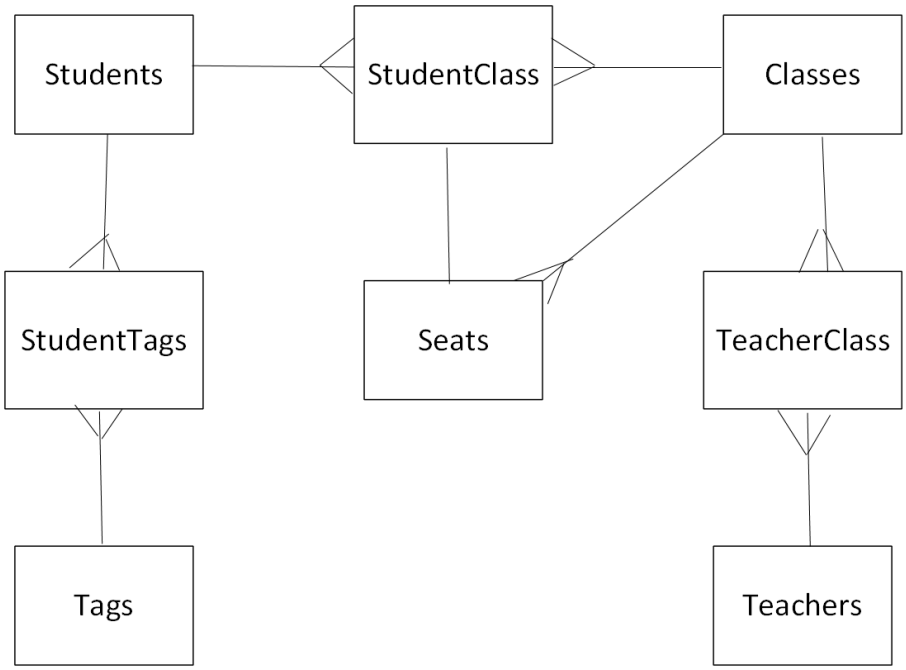


Figure 5: Entity relationship diagram between normalised databases

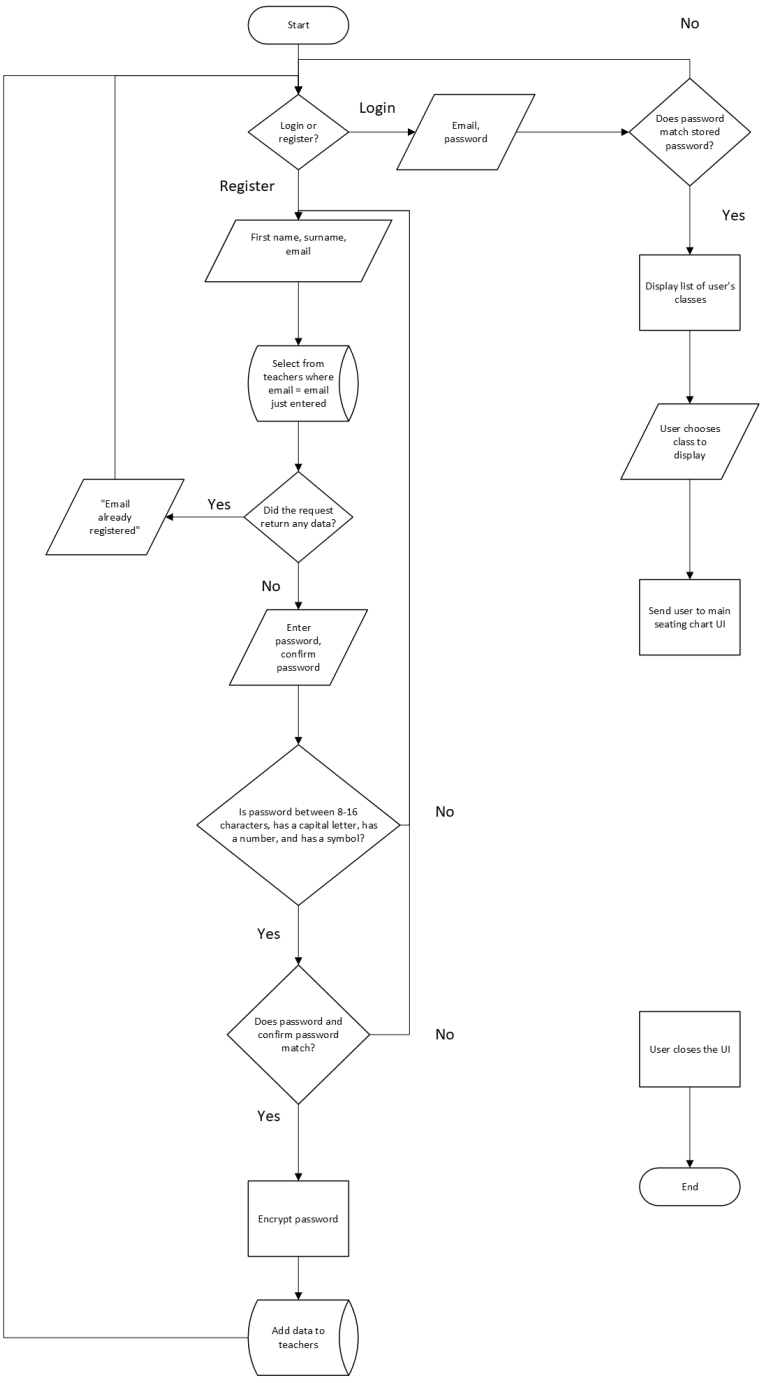


Figure 6: Flowchart for the logging in/registering process

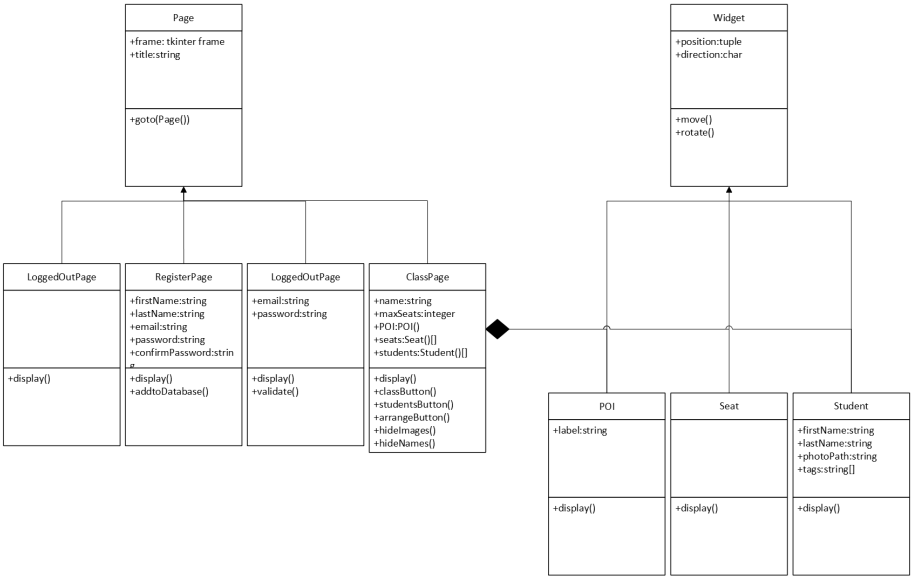


Figure 7: Class diagram showing the relationships between classes, attributes, and methods

3 Technical solutions and evaluations

Contents

3.1 19th May 9
3.2 4th June 16

3.1 19th May

I have added: databases in 3rd normal form (objective 5 done); account registration, with it saving the inputted details to a database after validation and password encryption (objective 1 done); login (objective 2 done); outline of the main seating chart page and menu bar (starting objective 4).

```
import tkinter as tk
from tkinter import ttk
import sqlite3 as sql
import bcrypt #Encrypt password
import re #Regex

#Constants

#Email validation
DOMAINS=("exe-coll.ac.uk","exetermathematicsschool.ac.uk")
#Regex for at least 1 letter, then any number of numbers, then one or no (+, least one alphanumeric character), then one @
#eg. EdHorncastle183+f8h_6f@
EMAILFORMAT="[A-z]+\d*(\+\w+)?@"
#Database path
DATABASE = "seatingChartData.db"

#Classes

#Base class for all page classes to inherit from
class Page:
    def __init__(self,root):
        self.root=root
        self.frame = ttk.Frame(self.root)
        self.frame.grid()
    def goto(self, page):
        self.frame.destroy()
        page(self.root)


#Page that is visible on first load
class LoggedOutPage(Page):
    def __init__(self,root):
        super().__init__(root)

```

```
35          ttk.Label(self.frame, text="Please register or log in to
    ↪ access your seating charts").grid(column=1,row=0)
36          ttk.Button(self.frame, text="Register", command=lambda:self.
    ↪ goto(RegisterPage)).grid(column=1,row=1)
37          ttk.Button(self.frame, text="Log in", command=lambda:self.
    ↪ goto(LoginPage)).grid(column=1,row=2)
38
39  #Page for registering new users, validating their inputs, and adding
    ↪ to database
40  class RegisterPage(Page):
41      def __init__(self,root):
42          super().__init__(root)
43
44          self.firstName = tk.StringVar()
45          self.lastName= tk.StringVar()
46          self.email = tk.StringVar()
47          self.password = tk.StringVar()
48          self.password2 = tk.StringVar()
49          self.invalid = tk.StringVar()
50
51          ttk.Button(self.frame, text="< Back", command=lambda:self.
    ↪ goto(LoggedOutPage)).grid(column=0,row=0)
52          ttk.Label(self.frame,text="Register:").grid(column=1,row=0)
53          ttk.Label(self.frame, text="First name:").grid(column=1,row=1
    ↪ ,padx=10,pady=10)
54          ttk.Entry(self.frame, textvariable = self.firstName).grid(
    ↪ column=2,row=1,columnspan=2)
55          ttk.Label(self.frame, text="Last name:").grid(column=1,row=2)
56          ttk.Entry(self.frame, textvariable=self.lastName).grid(column
    ↪ =2,row=2,columnspan=2)
57          ttk.Label(self.frame, text="Email:").grid(column=1,row=3)
58          ttk.Entry(self.frame, textvariable=self.email).grid(column=2,
    ↪ row=3,columnspan=2)
59          ttk.Label(self.frame, text="Password:").grid(column=1,row=4)
60          ttk.Entry(self.frame, textvariable=self.password,show='*').
    ↪ grid(column=2,row=4,columnspan=2)
61          ttk.Label(self.frame,text="Passwords must contain at least
    ↪ one each of capital letter, symbol, and number and be between
    ↪ 8-16 characters"
62                      ).grid(column=1,row=5,columnspan=10)
63          ttk.Label(self.frame, text="Confirm password:").grid(column=1
    ↪ ,row=6)
64          ttk.Entry(self.frame, textvariable=self.password2,show='*').
    ↪ grid(column=2,row=6,columnspan=2)
65          ttk.Button(self.frame, text="Register",command=self.
    ↪ addtoDatabase).grid(column=1,row=7)
66          ttk.Label(self.frame, textvariable=self.invalid).grid(column=
    ↪ 1,row=8,columnspan=10)
67
68      def addtoDatabase(self):
```

```
69          query="INSERT INTO Teachers (FirstName, LastName, Email,
    ↪ Password) VALUES (?, ?, ?, ?)"
70          #Checking email is at one of the allowed domains and follows
    ↪ the regex format at the top
71          validEmail=False
72          for domain in DOMAINS:
73              if re.search(EMAILFORMAT+domain,self.email.get()):
74                  validEmail=True
75          if validEmail:
76              #Checking password fields are valid
77              password=self.password.get()
78              if password==self.password2.get() and password!="":
79                  #At least one each of symbol, capital letter, and
    ↪ number between 8-16 characters
80                  if re.search("[^\w\s]+",password) and re.search("[A-Z
    ↪ ]+",password) and re.search("\d+",
81                  password) and len(password)>=8 and len(password)<=16:
82                      self.password = bcrypt.hashpw(password.encode('
    ↪ utf-8') , bcrypt.gensalt() )
83                      #Prevents emails being registered multiple times
84                      try:
85                          #Adding to the database
86                          with sql.connect(DATABASE) as conn:
87                              cur = conn.cursor()
88                              cur.execute(query,(self.firstName.get(),
    ↪ self.lastName.get(),self.email.get(),self.password))
89                              conn.commit()
90                          self.goto(ClassPage)
91              #Error messages
92                      except sql.IntegrityError:
93                          self.invalid.set("Email already registered")
94                      except:
95                          self.invalid.set("An error occurred")
96                  else:
97                      self.invalid.set("Password must contain at least
    ↪ one each of symbol, \ncapital letter, and number and must be
    ↪ between \n8-16 characters")
98              else:
99                  if password=="":
100                     self.invalid.set("Enter a password")
101                 elif password!=self.password2.get():
102                     self.invalid.set("Passwords do not match")
103         else:
104             self.invalid.set("Invalid email")
105
106 #Page for logging in existing users. Fetches password from database.
107 class LoginPage(Page):
108     def __init__(self,root):
109         super().__init__(root)
110
```

```
111         self.email = tk.StringVar()
112         self.password = tk.StringVar()
113         self.invalid=tk.StringVar()
114         #Add styling
115         ttk.Button(self.frame, text="< Back", command=lambda:self.
    ↪ goto(LoggedOutPage)).grid(column=0,row=0)
116         ttk.Label(self.frame,text="Log in:").grid(column=1,row=0)
117         ttk.Label(self.frame, text="Email:").grid(column=1,row=1)
118         ttk.Entry(self.frame, textvariable=self.email).grid(column=2,
    ↪ row=1,columnspan=2)
119         ttk.Label(self.frame, text="Password:").grid(column=1,row=2)
120         ttk.Entry(self.frame, textvariable=self.password,show='*').
    ↪ grid(column=2,row=2,columnspan=2)
121         ttk.Button(self.frame, text="Login",command=self.validate).
    ↪ grid(column=1,row=3)
122         ttk.Label(self.frame, textvariable=self.invalid).grid(column=
    ↪ 1,row=4)
123     def validate(self):
124         #Fetches encrypted correct password from databse
125         with sql.connect(DATABASE) as conn:
126             cur = conn.cursor()
127             cur.execute("SELECT Password from Teachers WHERE Email =
    ↪ ?",(self.email.get(),))
128             correctPass=cur.fetchone()
129         #correctPass is NoneType when the email entered is not in the
    ↪  database
130         try:
131             correctPass=correctPass[0]
132             #Checks that the password entered matches the encrypted
    ↪ correct password
133             if bcrypt.checkpw(self.password.get().encode('utf-8'),
    ↪ correctPass):
134                 self.goto(ClassPage)
135         #Error messages
136             else:
137                 self.invalid.set("Password incorrect.")
138         except:
139             self.invalid.set("Email is not registered.")
140
141
142
143 #Page for displaying the seating chart and menu of tools.
144 class ClassPage(Page):
145     def __init__(self,root):
146         super().__init__(root)
147
148         self.CANVASWIDTH = 900
149         self.CANVASHEIGHT = 900
150
151         self.displayMenu()
```

```
152         tk.Canvas(self.frame,
153                         width=self.CANVASWIDTH,
154                         height=self.CANVASHEIGHT,
155                         background='white').grid(column=0,row=1,
    ↪ columnspan=5,rowspan=5)
156 
157     def displayMenu(self):
158         menubar = tk.Menu(self.root,background="lightyellow")
159         self.root['menu'] = menubar
160 
161         #"Class" dropdown menu
162         menu_class = tk.Menu(menubar)
163         menubar.add_cascade(menu=menu_class, label='Class')
164         menu_class.add_command(label='Switch class', command="")
165         menu_class.add_command(label="Save", command="")
166         menu_class.add_command(label="Edit class",command="")
167 
168         #"Students" dropdown menu
169         menu_students = tk.Menu(menubar)
170         menubar.add_cascade(menu=menu_students, label="Students")
171         menu_students.add_command(label='Edit students', command ="")
172 
173         #"Arrange" dropdown menu
174         menu_arrange = tk.Menu(menubar)
175         menubar.add_cascade(menu=menu_arrange, label="Arrange")
176 
177         #Tick box to choose to hide or show names of students
178         hideNames=tk.StringVar()
179         menubar.add_checkbutton(label="Hide names", variable=
    ↪ hideNames, onvalue=True, offvalue=False)
180 
181         #Tick box to choose to hide or show images of students
182         hideImages=tk.StringVar()
183         menubar.add_checkbutton(label="Hide images", variable=
    ↪ hideImages, onvalue=True, offvalue=False)
184 
185 
186 #Base class for all widgets to inherit from
187 class Widget:
188     def __init__(self,x=0,y=0,rot='S'):
189         self.x=x
190         self.y=y
191         self.dir=rot
192     def move(self):
193         print("uyp") #
194     def rotate(self):
195         print("yup") #
196 
197 class Seat(Widget):
198     def __init__(self):
```

```
199         super.__init__()
200 
201 class POI(Widget):
202     def __init__(self):
203         super.__init__()
204 
205 class Student(Widget):
206     def __init__(self):
207         super.__init__()
208     def getData(self,id):
209         #
    ↪ <<<<------------------------------------------------------------
210         with sql.connect(DATABASE) as conn:
211             cur = conn.cursor()
212             cur.execute("SELECT FirstName, LastName, PhotoPath FROM
    ↪ Students WHERE StudentID = ?",(id,))
213             data=cur.fetchone()
214         self.firstName=data[0]
215         self.lastName=data[1]
216         self.photoPath=data[2]
217         #Tags
218 
219 #Subroutines
220 
221 def main():
222     root = tk.Tk()
223     root.title( "Seating Planner" )
224     root.option_add('*tearOff', False)
225     LoggedOutPage(root)
226 
227     root.mainloop()
228 
229 #=============================================
230 main()
231 
```

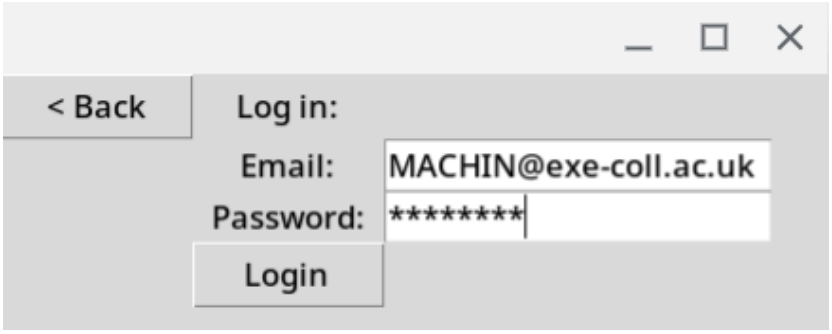


Figure 8: Login page



Figure 9: Main class page after login/register

3.2 4th June

```
import tkinter as tk
from tkinter import ttk
from PIL import Image,ImageTk
import sqlite3 as sql
import bcrypt #Encrypt password
import re #Regex

#Constants

#Email validation
DOMAINS=("exe-coll.ac.uk","exetermathematicsschool.ac.uk")
#Regex for at least 1 letter, then any number of numbers, then one or no (+, least one alphanumeric character), then one @
#eg. EdHorncastle183+f8h_6f@
EMAILFORMAT="[A-z]+\d*(\+\w+)?@"
#Database path
DATABASE = "seatingChartData.db"

#Classes

#Base class for all page classes to inherit from
class Page:
    def __init__(self,root:tk.Tk):
        self.root=root
        self.frame = ttk.Frame(self.root)
        self.frame.grid()
    def goto(self, page):
        self.frame.destroy()
        page(self.root)


#Page that is visible on first load
class LoggedOutPage(Page):
    def __init__(self,root):
        super().__init__(root)

        ttk.Label(self.frame, text="Please register or log in to access your seating charts").grid(column=1,row=0)
        ttk.Button(self.frame, text="Register", command=lambda:self.goto(RegisterPage)).grid(column=1,row=1)
        ttk.Button(self.frame, text="Log in", command=lambda:self.goto(LoginPage)).grid(column=1,row=2)

#Page for registering new users, validating their inputs, and adding to database
class RegisterPage(Page):
    def __init__(self,root):
```

```
43          super().__init__(root)
44
45          self.firstName = tk.StringVar()
46          self.lastName= tk.StringVar()
47          self.email = tk.StringVar()
48          self.password = tk.StringVar()
49          self.password2 = tk.StringVar()
50          self.invalid = tk.StringVar()
51
52          ttk.Button(self.frame, text="< Back", command=lambda:self.
    ↪ goto(LoggedOutPage)).grid(column=0,row=0)
53          ttk.Label(self.frame,text="Register:").grid(column=1,row=0)
54          ttk.Label(self.frame, text="First name:").grid(column=1,row=1
    ↪ ,padx=10,pady=10)
55          ttk.Entry(self.frame, textvariable = self.firstName).grid(
    ↪ column=2,row=1,columnspan=2)
56          ttk.Label(self.frame, text="Last name:").grid(column=1,row=2)
57          ttk.Entry(self.frame, textvariable=self.lastName).grid(column
    ↪ =2,row=2,columnspan=2)
58          ttk.Label(self.frame, text="Email:").grid(column=1,row=3)
59          ttk.Entry(self.frame, textvariable=self.email).grid(column=2,
    ↪ row=3,columnspan=2)
60          ttk.Label(self.frame, text="Password:").grid(column=1,row=4)
61          ttk.Entry(self.frame, textvariable=self.password,show='*').
    ↪ grid(column=2,row=4,columnspan=2)
62          ttk.Label(self.frame,text="Passwords must contain at least
    ↪ one each of capital letter, symbol, and number and be between
    ↪ 8-16 characters"
63                      ).grid(column=1,row=5,columnspan=10)
64          ttk.Label(self.frame, text="Confirm password:").grid(column=1
    ↪ ,row=6)
65          ttk.Entry(self.frame, textvariable=self.password2,show='*').
    ↪ grid(column=2,row=6,columnspan=2)
66          ttk.Button(self.frame, text="Register",command=self.
    ↪ addtoDatabase).grid(column=1,row=7)
67          ttk.Label(self.frame, textvariable=self.invalid).grid(column=
    ↪ 1,row=8,columnspan=10)
68
69      def addtoDatabase(self):
70          query="INSERT INTO Teachers (FirstName, LastName, Email,
    ↪ Password) VALUES (?, ?, ?, ?)"
71          #Checking email is at one of the allowed domains and follows
    ↪ the regex format at the top
72          validEmail=False
73          for domain in DOMAINS:
74              if re.search(EMAILFORMAT+domain,self.email.get()):
75                  validEmail=True
76          if validEmail:
77              #Checking password fields are valid
78              password=self.password.get()
```

```
79              if password==self.password2.get() and password!="":
80                  #At least one each of symbol, capital letter, and
    ↪ number between 8-16 characters
81                  if re.search("[^\w\s]+",password) and re.search("[A-Z
    ↪ ]+",password) and re.search("\d+",
82                  password) and len(password)>=8 and len(password)<=16:
83                      self.password = bcrypt.hashpw(password.encode('
    ↪ utf-8') , bcrypt.gensalt() )
84                      #Prevents emails being registered multiple times
85                      try:
86                          #Adding to the database
87                          with sql.connect(DATABASE) as conn:
88                              cur = conn.cursor()
89                              cur.execute(query,(self.firstName.get(),
    ↪ self.lastName.get(),self.email.get(),self.password))
90                              conn.commit()
91                          self.goto(ClassPage)
92              #Error messages
93                      except sql.IntegrityError:
94                          self.invalid.set("Email already registered")
95                      except:
96                          self.invalid.set("An error occurred")
97                  else:
98                      self.invalid.set("Password must contain at least
    ↪ one each of symbol, \ncapital letter, and number and must be
    ↪ between \n8-16 characters")
99              else:
100                 if password=="":
101                     self.invalid.set("Enter a password")
102                 elif password!=self.password2.get():
103                     self.invalid.set("Passwords do not match")
104         else:
105             self.invalid.set("Invalid email")
106
107     def validate(self):
108         ''''''
109
110
111 #Page for logging in existing users. Fetches password from database.
112 class LoginPage(Page):
113     def __init__(self,root):
114         super().__init__(root)
115
116         self.email = tk.StringVar()
117         self.password = tk.StringVar()
118         self.invalid=tk.StringVar()
119         #Add styling
120         ttk.Button(self.frame, text="< Back", command=lambda:self.
    ↪ goto(LoggedOutPage)).grid(column=0,row=0)
121         ttk.Label(self.frame,text="Log in:").grid(column=1,row=0)
```

```
122         ttk.Label(self.frame, text="Email:").grid(column=1,row=1)
123         ttk.Entry(self.frame, textvariable=self.email).grid(column=2,
    ↪ row=1,columnspan=2)
124         ttk.Label(self.frame, text="Password:").grid(column=1,row=2)
125         ttk.Entry(self.frame, textvariable=self.password,show='*').
    ↪ grid(column=2,row=2,columnspan=2)
126         ttk.Button(self.frame, text="Login",command=self.validate).
    ↪ grid(column=1,row=3)
127         ttk.Label(self.frame, textvariable=self.invalid).grid(column=
    ↪ 1,row=4)
128     def validate(self):
129         #Fetches encrypted correct password from databse
130         with sql.connect(DATABASE) as conn:
131             cur = conn.cursor()
132             cur.execute("SELECT Password from Teachers WHERE Email =
    ↪ ?",(self.email.get(),))
133             correctPass=cur.fetchone()
134         #correctPass is NoneType when the email entered is not in the
    ↪  database
135         try:
136             correctPass=correctPass[0]
137             #Checks that the password entered matches the encrypted
    ↪ correct password
138             if bcrypt.checkpw(self.password.get().encode('utf-8'),
    ↪ correctPass):
139                 self.goto(ClassPage)
140         #Error messages
141             else:
142                 self.invalid.set("Password incorrect.")
143         except:
144             self.invalid.set("Email is not registered.")
145
146
147
148 #Page for displaying the seating chart and menu of tools.
149 class ClassPage(Page):
150     def __init__(self,root):
151         super().__init__(root)
152
153         self.classid=1 #Change when different class added
154
155         self.CANVASWIDTH =  900
156         self.CANVASHEIGHT =  900
157
158         self.displayMenu()
159         self.canvas = tk.Canvas(self.frame,
160                                 width=self.CANVASWIDTH,
161                                 height=self.CANVASHEIGHT,
162                                 background='white')
163         self.canvas.grid(column=0,row=1,columnspan=5,rowspan=5)
```

```
164         maxSeats=10
165         self.seats=list()
166         for i in range(maxSeats):
167             self.seats.append(Seat(self.canvas,x=i*(self.CANVASWIDTH/
    ↪ maxSeats)))
168         self.students=self.getStudentData()
169         for i in range(len(self.students)):
170             self.students[i]=Student(self.canvas,self.students[i],i*(
    ↪ self.CANVASWIDTH/len(self.students)),100,seats=self.seats)
171 
172     def getStudentData(self):
173         query="SELECT StudentID, FirstName, LastName, PhotoPath FROM
    ↪ Students INNER JOIN StudentClass USING (StudentID) WHERE
    ↪ ClassID = ?"
174         with sql.connect(DATABASE) as conn:
175             cur = conn.cursor()
176             cur.execute(query,(self.classid,))
177             students=cur.fetchall()
178             return students
179 
180     def displayMenu(self):
181         menubar = tk.Menu(self.root,background="lightyellow")
182         self.root['menu'] = menubar
183 
184         #"Class" dropdown menu
185         menu_class = tk.Menu(menubar)
186         menubar.add_cascade(menu=menu_class, label='Class')
187         menu_class.add_command(label='Switch class', command="")
188         menu_class.add_command(label="Save", command="")
189         menu_class.add_command(label="Edit class",command="")
190 
191         #"Students" dropdown menu
192         menu_students = tk.Menu(menubar)
193         menubar.add_cascade(menu=menu_students, label="Students")
194         menu_students.add_command(label='Edit students', command ="")
195 
196         #"Arrange" dropdown menu
197         menu_arrange = tk.Menu(menubar)
198         menubar.add_cascade(menu=menu_arrange, label="Arrange")
199 
200         #Tick box to choose to hide or show names of students
201         self.hideNames=False
202         menubar.add_checkbutton(label="Hide names", command=self.
    ↪ changeNameVisibility)
203 
204         #Tick box to choose to hide or show images of students
205         self.hideImages=tk.StringVar()
206         menubar.add_checkbutton(label="Hide images", variable=self.
    ↪ hideImages, onvalue=True, offvalue=False,command="")
207 
```

```
208         #menubar.add_command(label="Add seat")
209 
210     def changeNameVisibility(self):
211         if self.hideNames==False:
212             self.hideNames=True
213             self.canvas.itemconfigure("studentname",text="")
214         else:
215             self.hideNames=False
216             for student in self.students:
217                 self.canvas.itemconfigure(student.textid,text=student
    ↪ .firstName+" "+student.lastName)
218 
219 
220 
221 #Base class for all widgets to inherit from
222 class Widget:
223     def __init__(self,canvas:tk.Canvas,x,y,width,height):
224         self.x=x
225         self.y=y
226         self.canvas=canvas
227         self.width=width
228         self.height=height
229     def move(self,event:tk.Event):
230         #Prevents widget being moved outside of canvas
231         if event.x<int(self.canvas["width"]) and event.y<int(self.
    ↪ canvas["height"]) and event.x>0 and event.y>0:
232             #Moves so that the mouse is the center
233             self.canvas.move(self.tag,event.x-self.x-self.width/2,
    ↪ event.y-self.y-self.width/2)
234             self.x=self.canvas.coords(self.id)[0]
235             self.y=self.canvas.coords(self.id)[1]
236 
237 
238 class Seat(Widget):
239     #Keeping track of total seats in the class to prevent more seats
    ↪ than maxSeats in database from being created
240     seats=0
241     def __init__(self,canvas,x=0,y=0,rot='S',width=100):
242         super().__init__(canvas,x,y,width,width)
243         Seat.seats+=1
244 
245         self.dir=rot
246 
247         self.id=self.canvas.create_rectangle(self.x,self.y,self.x+
    ↪ self.width,self.y +self.width, fill='red', outline='blue',tags
    ↪ =('seat','seatbox'))
248         #Rectangles and arrows have the same tag so they move
    ↪ together when dragged
249         self.tag="seat"+str(self.id)
250         self.canvas.addtag_withtag(self.tag,self.id)
```

```
251
252         self.textid=self.canvas.create_text(self.x,self.y,tags=('seat
    ↪ ',self.tag),font=("Arial", 20))
253         self.displayDirection()
254
255         self.canvas.tag_bind(self.tag,"<B1-Motion>", self.move) #Hold
    ↪  left click and drag to move widget
256         self.canvas.tag_bind(self.tag,"<ButtonPress-3>",self.rotate)
    ↪ #Press right click to rotate arrow
257
258     def rotate(self,*args):
259         match self.dir:
260             case 'W':
261                 self.dir='N'
262             case 'N':
263                 self.dir='E'
264             case 'S':
265                 self.dir='W'
266             case 'E':
267                 self.dir='S'
268         self.displayDirection()
269
270     def displayDirection(self,*args):
271         match self.dir:
272             case 'N':
273                 self.canvas.itemconfigure(self.textid,text='↑')
274                 self.canvas.coords(self.textid, self.x+self.width/2,
    ↪ self.y)
275             case 'E':
276                 self.canvas.itemconfigure(self.textid,text='→')
277                 self.canvas.coords(self.textid, self.x+self.width,
    ↪ self.y+self.width/2)
278             case 'W':
279                 self.canvas.itemconfigure(self.textid,text='←')
280                 self.canvas.coords(self.textid, self.x, self.y+self.
    ↪ width/2)
281             case 'S':
282                 self.canvas.itemconfigure(self.textid,text='↓')
283                 self.canvas.coords(self.textid, self.x+self.width/2,
    ↪ self.y+self.width)
284             case _:
285                 self.canvas.itemconfigure(self.textid,text='↓')
286                 self.canvas.coords(self.textid, self.x+self.width/2,
    ↪ self.y+self.width)
287                 self.dir='S'
288
289
290
291 class POI(Widget):
292     def __init__(self):
```

```
293         super().__init__()
294 
295 class Student(Widget):
296     def __init__(self,canvas,data,x=0,y=0,width=75,seats:Seat=None):
297         super().__init__(canvas,x,y,width,width)
298 
299         self.studentID=data[0]
300         self.firstName=data[1]
301         self.lastName=data[2]
302         self.photoPath=data[3]
303 
304         self.seat=None
305         self.seats=seats
306 
307         self.id=self.canvas.create_rectangle(self.x,self.y,self.x+
    ↪ self.width,self.y +self.width, fill='#ffdb32', outline='black'
    ↪ ,tags='student')
308         self.tag="student"+str(self.id)
309         self.canvas.addtag_withtag(self.tag,self.id)
310 
311         self.textid=self.canvas.create_text(self.x+self.width,self.y,
    ↪ tags=('student',self.tag,"studentname"),text=self.firstName+'
    ↪ '+self.lastName,anchor=tk.NE,width=self.width,justify='right')
312 
313         self.canvas.tag_bind(self.tag,"<B1-Motion>", self.move)
314         self.canvas.tag_bind(self.tag,"<ButtonRelease-1>", self.
    ↪ checkForSeat)
315         self.canvas.tag_bind(self.tag,"<ButtonPress-3>",self.
    ↪ rotateSeat)
316 
317     def checkForSeat(self,event):
318         seatFound=False
319         for seatID in self.canvas.find_withtag('seatbox'): #Find if
    ↪ student is entirely within a seat
320             seatCoords=self.canvas.coords(seatID)
321             ids=self.canvas.find_enclosed(seatCoords[0],seatCoords[1
    ↪ ],seatCoords[2],seatCoords[3])
322             if self.id in ids: #Save seat id and snap student to
    ↪ centre of seat
323                 self.seat=seatID
324                 seatInfo=self.canvas.itemconfigure(self.seat)
325                 seatFound=True
326 
327         if isinstance(self.seat,int) and not seatFound: #When moved
    ↪ off seat, remove seat id
328             self.seat=None
329 
330     def rotateSeat(self,*args):
331         if isinstance(self.seat,int):
332             self.seats[self.seat//2].rotate()
```

```
333
334 #Subroutines
335
336 def main():
337     root = tk.Tk()
338     root.title( "Seating Planner" )
339     root.option_add('*tearOff', False)
340     LoggedOutPage(root)
341
342     root.mainloop()
343
344 #=============================================
345 main()
```

4 Testing